GitHub - 使用生成式 AI 自動執行程式碼審查作業

來源：https://codelabs.developers.google.com/genai-for-dev-github-code-review?hl=zh-tw

步驟數：14

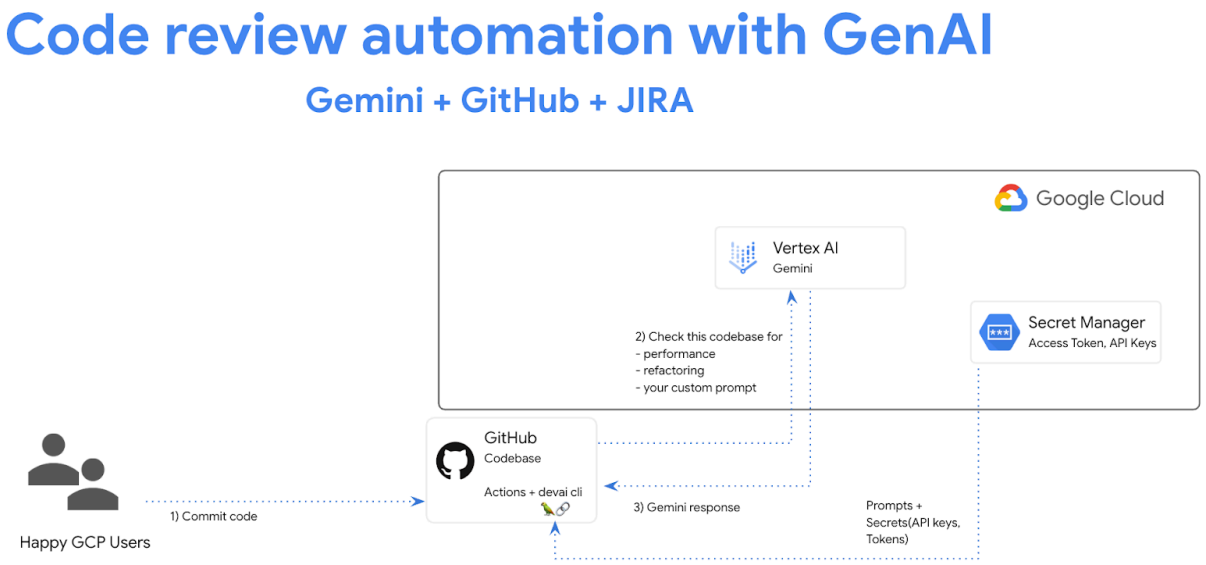
GitHub - 使用 GenAI 自動執行程式碼審查

目錄

步驟 1 · 約 0 分鐘

1. 總覽

在本實驗室中，您將設定 GitHub 動作工作流程，並與 Gemini 整合，自動執行程式碼審查步驟。



學習目標

在本實驗室中，您將瞭解如何執行下列操作：

- 如何在 GitHub 中新增生成式 AI 程式碼審查自動化步驟
- 如何在本機執行 devai CLI，自動執行程式碼審查

必要條件

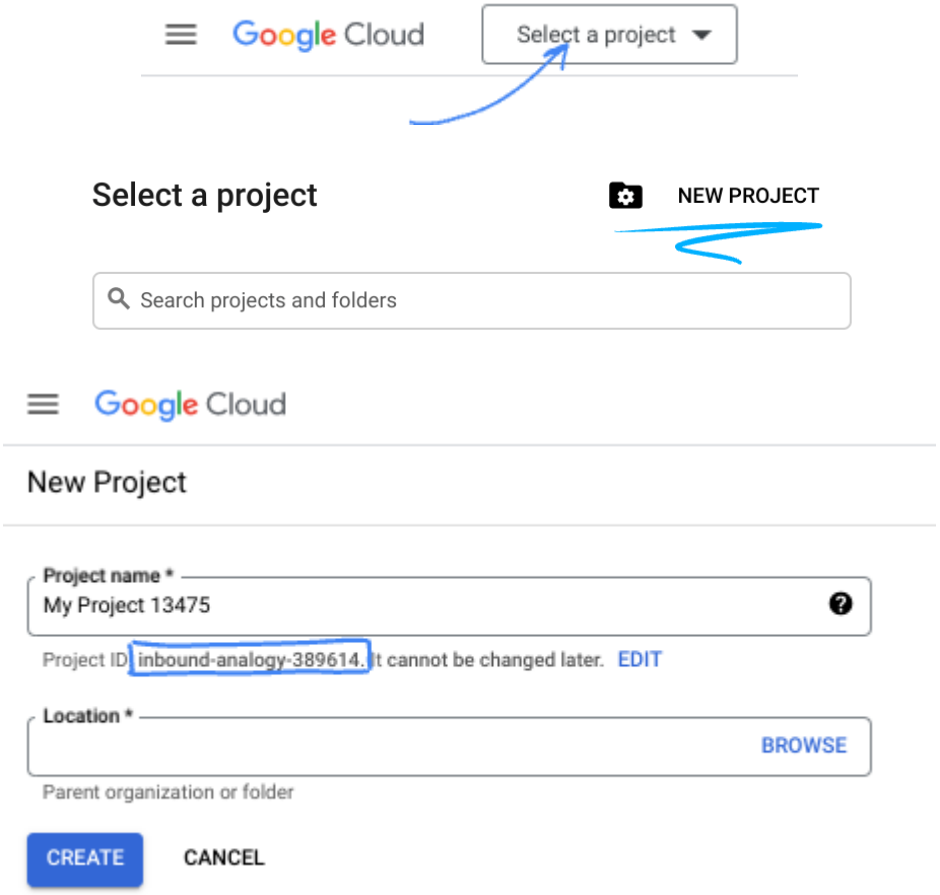
- 本實驗室假設您已熟悉 Cloud 控制台和 Cloud Shell 環境。

步驟 2 · 約 0 分鐘

2. 設定和需求

設定 Cloud 專案

1. 登入 Google Cloud 控制台，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先建立帳戶。



- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱說明文件。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

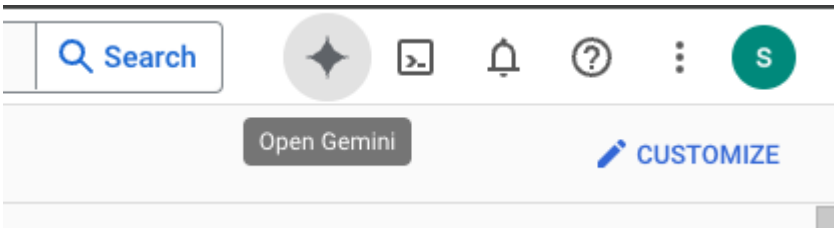
注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中啟用帳單，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

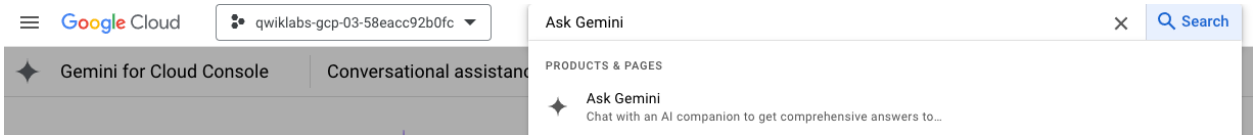
用者可參加價值$300 美元的免費試用計畫。

環境設定

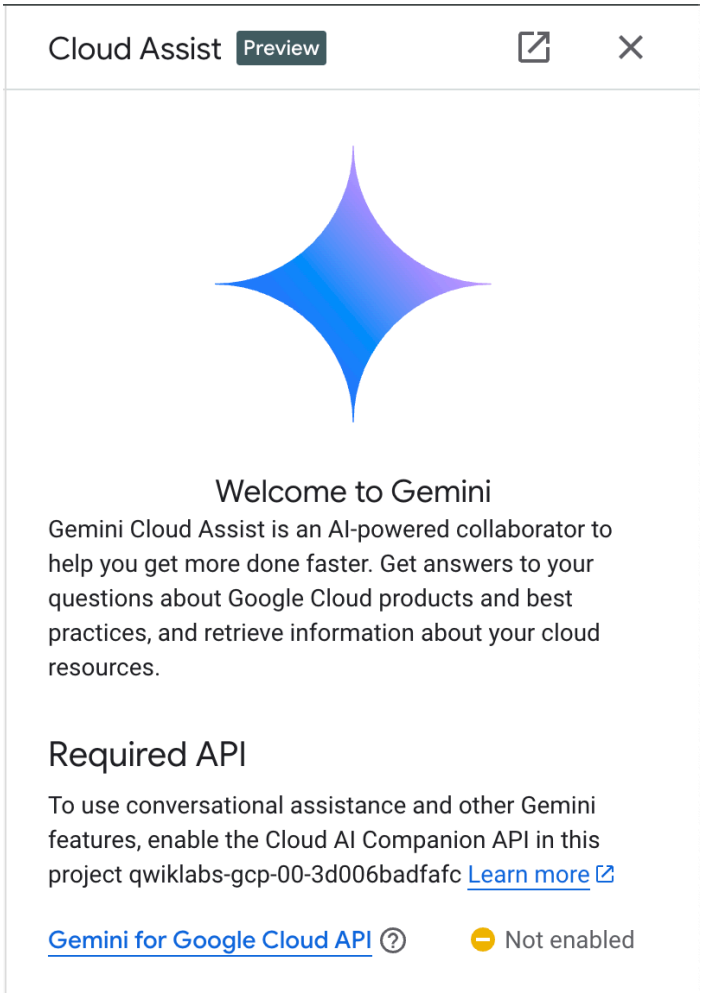
開啟 Gemini 對話。



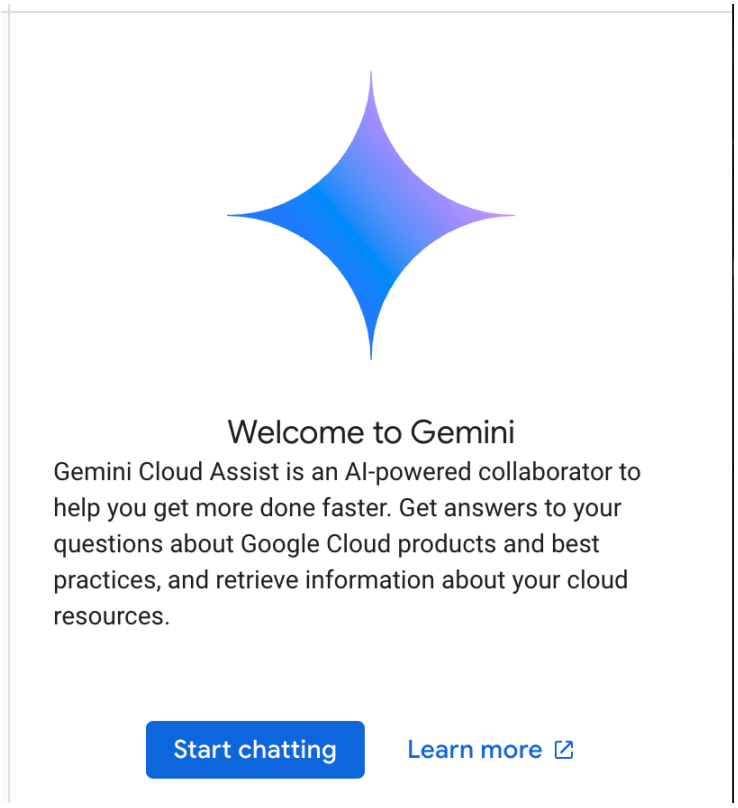
或在搜尋列中輸入「問問 Gemini」。



啟用 Gemini for Google Cloud API：



按一下「`Start chatting`」，然後按照其中一個範例問題操作，或自行輸入提示詞來試用。



建議提示詞：

- 請用 5 個重點說明 Cloud Run。
- 您是 Google Cloud Run 產品經理，請用 5 個簡短重點向學生說明 Cloud Run。
- 您是 Google Cloud Run 產品經理，請用 5 個簡短重點向認證 Kubernetes 開發人員說明 Cloud Run。
- 您是 Google Cloud Run 產品經理，請以 5 個簡短重點，向資深開發人員說明何時該使用 Cloud Run，何時該使用 GKE。

如要進一步瞭解如何撰寫更優質的提示，請參閱提示指南。

瞭解 Gemini for Google Cloud 如何使用您的資料

Google 對隱私權的承諾

Google 是業界首批發布 AI/機器學習隱私權承諾的公司，該文提到我們的信念：除了極致的安全性之外，客戶也應該對儲存在雲端的自家資料保有**最大的掌控權**。

您提交及收到的資料

您向 Gemini 提出的問題 (包括提交給 Gemini 分析或完成的任何輸入資訊或程式碼)，都稱為提示。您從 Gemini 取得的答案或程式碼完成建議稱為「回覆」。**我們不會將您的提示詞或 Gemini 的回覆用做模型訓練資料**。

提示加密

當您將提示提交給 Gemini 時，**您的資料會在傳輸過程中加密**，然後輸入 Gemini 的基礎模型。

Gemini 生成的節目資料

Gemini 是以 Google Cloud 第一方程式碼和精選第三方程式碼訓練而成。**您必須對程式碼的安全性、測試和效用負責**，包括 Gemini 提供的任何程式碼完成、生成或分析功能。

進一步瞭解 Google 如何處理提示。

步驟 3 · 約 0 分鐘

3. 測試提示的選項

如要變更/延長現有的 devai CLI 提示，有幾種做法可供選擇。

- Vertex AI Studio

Vertex AI Studio 是 Google Cloud Vertex AI 平台的一部分，專門用於簡化及加速生成式 AI 模型的開發和使用。

- Google AI Studio

Google AI Studio 是網頁工具，可讓您設計提示工程原型，並透過 Gemini API 進行實驗。

- Gemini 網頁應用程式 (gemini.google.com)

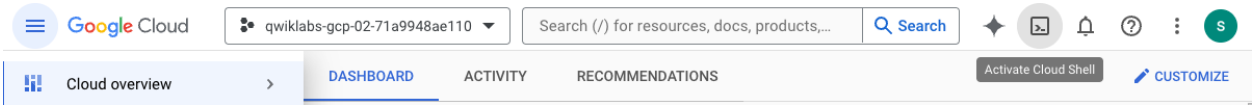
Google Gemini 網頁應用程式 (gemini.google.com) 是一款網頁工具，可協助您探索及運用 Google Gemini AI 模型強大的功能。

- Android和 iOS 版 Google 應用程式

步驟 4 · 約 0 分鐘

4. 建立服務帳戶

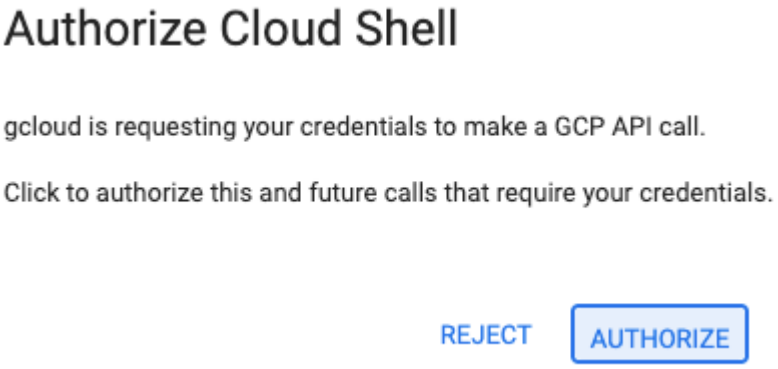
按一下搜尋列右側的圖示，啟用 Cloud Shell。



在開啟的終端機中，啟用使用 Vertex AI API 和 Gemini 對話 所需的服務。

```
gcloud services enable \
    aiplatform.googleapis.com \
    cloudaicompanion.googleapis.com \
    cloudresourcemanager.googleapis.com \
    secretmanager.googleapis.com
```

如果系統提示您授權，請點選「授權」繼續操作。



執行下列指令，建立新的服務帳戶和金鑰。

您將使用這個服務帳戶，透過 CICD 管道對 Vertex AI Gemini API 發出 API 呼叫。

```
PROJECT_ID=$(gcloud config get-value project)
SERVICE_ACCOUNT_NAME='vertex-client'
DISPLAY_NAME='Vertex Client'
KEY_FILE_NAME='vertex-client-key'

gcloud iam service-accounts create $SERVICE_ACCOUNT_NAME --display-name "$DISPLAY_NAME"
```

授予角色。

```
gcloud projects add-iam-policy-binding $PROJECT_ID --
member="serviceAccount:$SERVICE_ACCOUNT_NAME@$PROJECT_ID.iam.gserviceaccount.com" --
role="roles/aiplatform.admin" --condition None

gcloud projects add-iam-policy-binding $PROJECT_ID --
member="serviceAccount:$SERVICE_ACCOUNT_NAME@$PROJECT_ID.iam.gserviceaccount.com" --
role="roles/secretmanager.secretAccessor" --condition None

gcloud iam service-accounts keys create $KEY_FILE_NAME.json --iam-
account=$SERVICE_ACCOUNT_NAME@$PROJECT_ID.iam.gserviceaccount.com
```

步驟 5 · 約 0 分鐘

5. 將 GitHub 存放區分支新增至個人 GitHub 存放區

前往 https://github.com/GoogleCloudPlatform/genai-for-developers/fork，然後選取您的 GitHub 使用者 ID 做為擁有者。

請按一下 [`Create fork`]。

步驟 6 · 約 0 分鐘

6. 啟用 GitHub Actions 工作流程

在瀏覽器中開啟已分叉的 GitHub 存放區，然後切換至「`Actions`」分頁，啟用工作流程。

Workflows aren't being run on this forked repository

Because this repository contained workflow files when it was forked, we have disabled them from running on this fork. Make sure you understand the configured workflows and their expected usage before enabling Actions on this repository.

I understand my workflows, go ahead and enable them

View the workflows directory

步驟 7 · 約 0 分鐘

7. 新增存放區密鑰

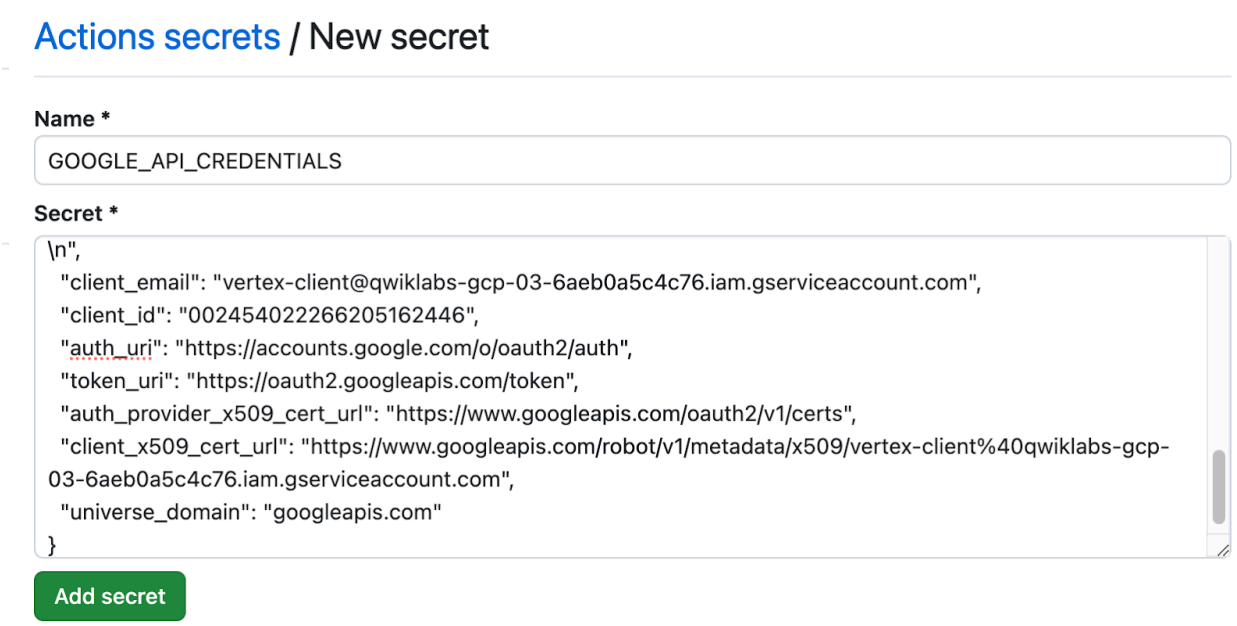
在建立分支的 GitHub 存放區中，於「`Settings / Secrets and variables / Actions`」下方建立存放區密鑰。

新增名為「`GOOGLE_API_CREDENTIALS`」的存放區密鑰。

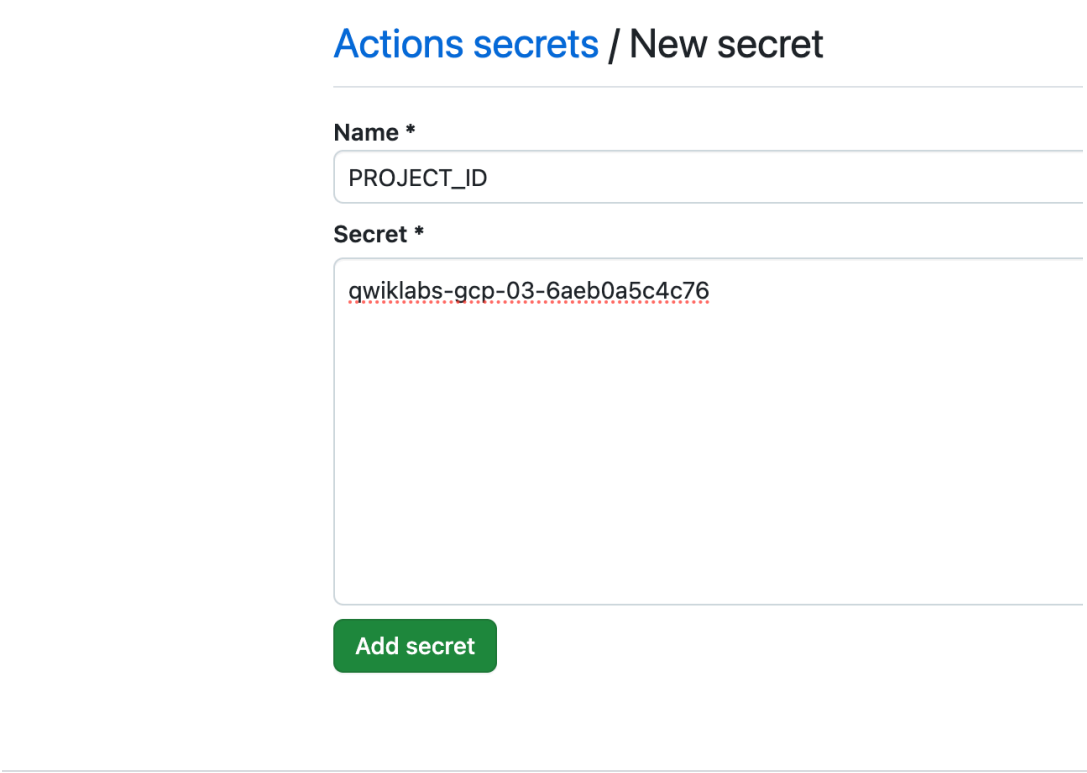
切換至 Google Cloud Shell 視窗/分頁，並在 Cloud Shell 終端機中執行下列指令。

```
cat ~/vertex-client-key.json
```

複製檔案內容，並貼上做為密鑰的值。



新增 `PROJECT_ID` 密鑰，並將 Qwiklabs 專案 ID 設為值：

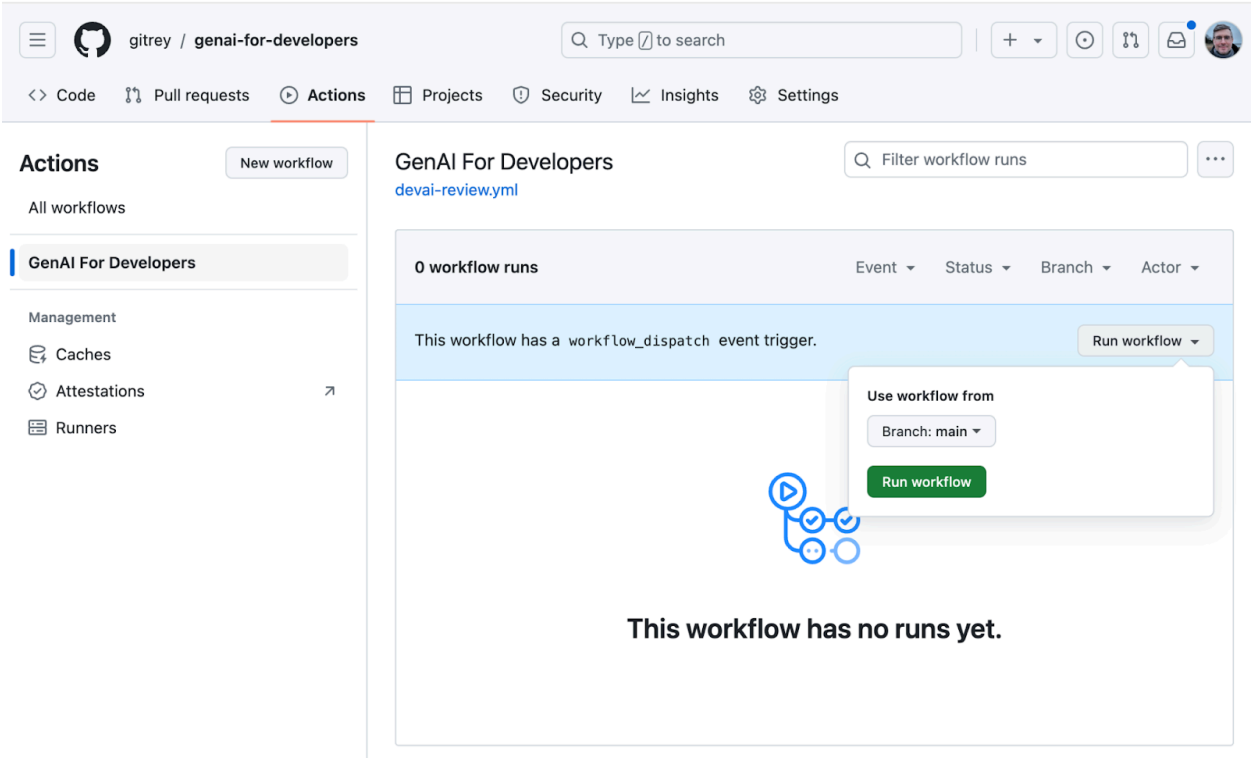

Actions secrets / New secret
Name *
PROJECT_ID
Secret *
qwiklabs-gcp-03-6aeb0a5c4c76
Add secret

步驟 8 · 約 0 分鐘

8. 執行 GitHub Actions 工作流程

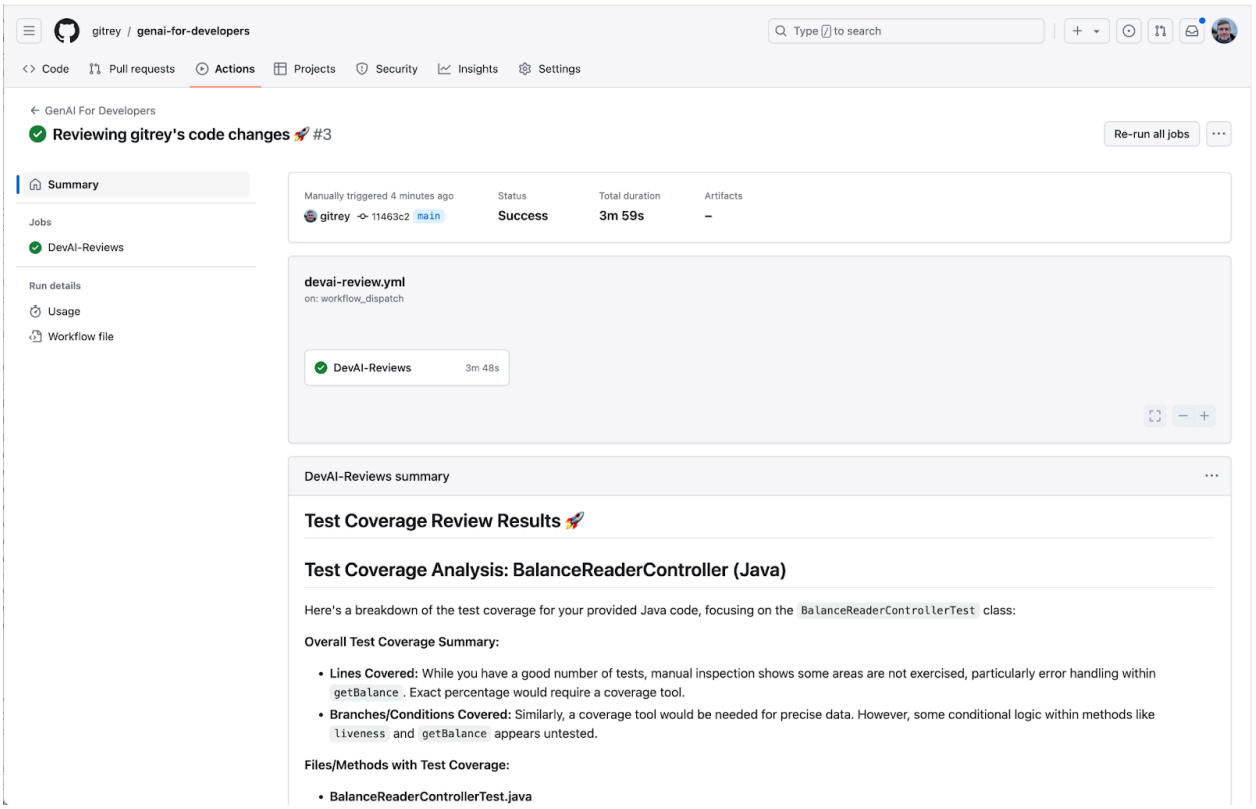
在瀏覽器中前往 GitHub 存放區，然後執行工作流程。

工作流程已設定為隨選執行。



步驟 9 · 約 0 分鐘

9. 查看工作流程輸出內容



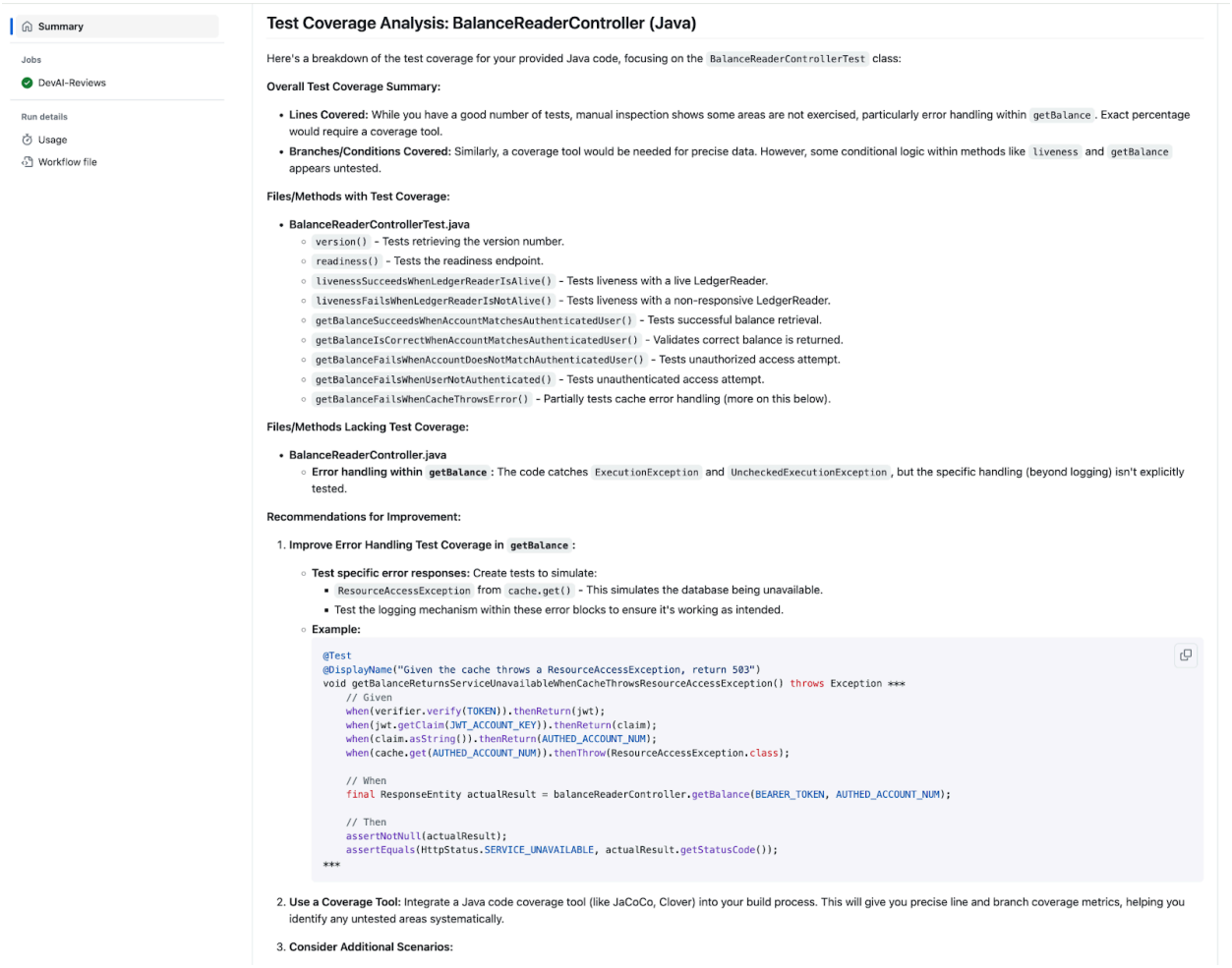
管道指令總覽

GitHub Actions 工作流程設定：devai-review.yml

測試涵蓋範圍指令的結果：

```
devai review testcoverage -c ${{ github.workspace }}/sample-
app/src/main/java/anthos/samples/bankofanthos/balancereader
```

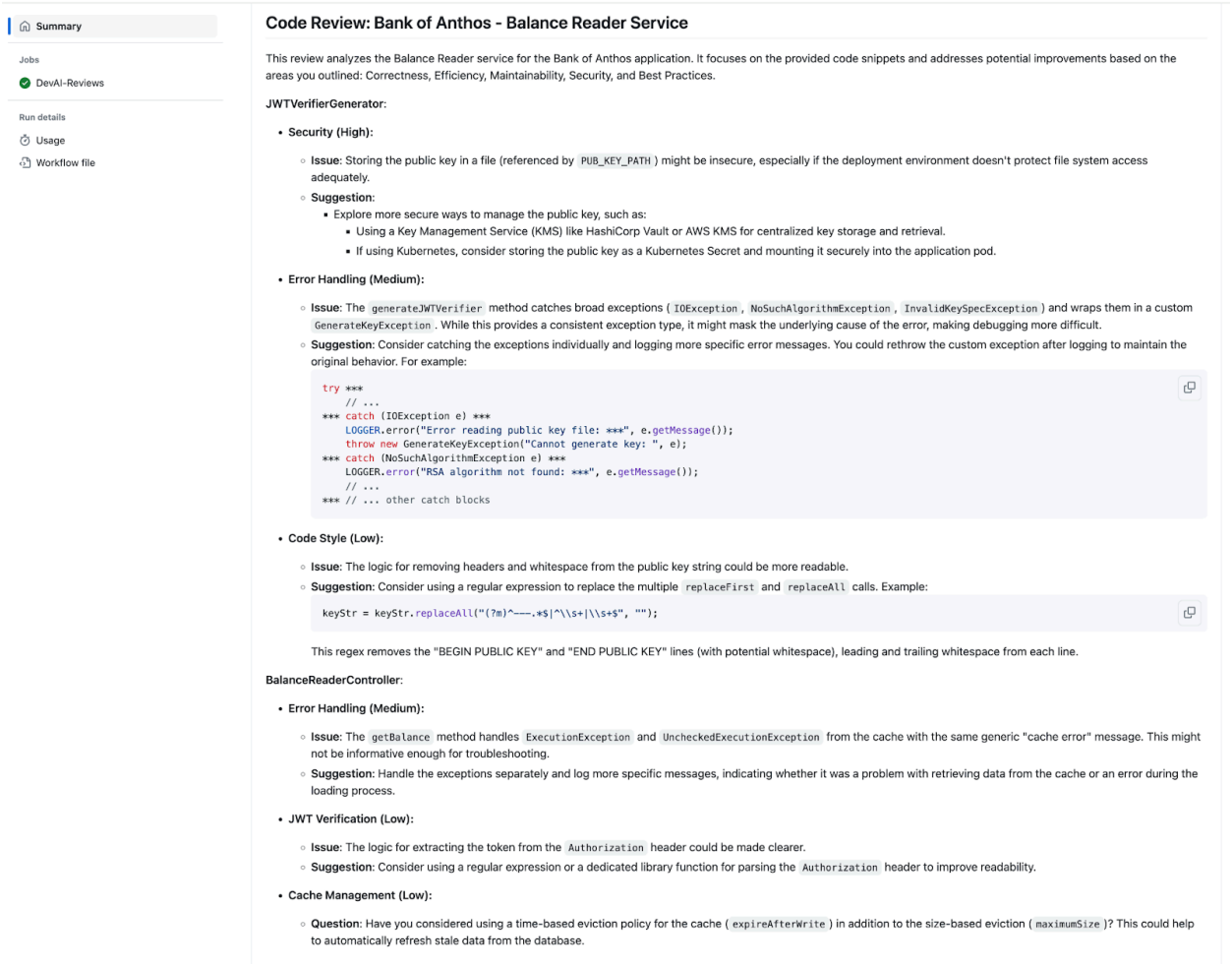
這項 `devai review testcoverage` 指令會使用生成式 AI 模型 Gemini，分析程式碼和相關聯的測試套件 (如有)。這項工具會評估所提供程式碼的測試涵蓋範圍，找出有單元測試和沒有單元測試的檔案和方法。然後，這項指令會運用模型提供涵蓋範圍摘要，包括涵蓋的程式碼行和分支/條件等指標。根據分析結果，這項工具會提供建議，協助您改善測試涵蓋範圍、建議要新增的特定測試，以及提供測試最佳做法的一般建議。最後，透過指令列輸出 Gemini 模型的相關回覆，包括缺少涵蓋範圍的檔案，以及改善現有測試的建議。



程式碼審查指令的結果：

```
devai review code -c ${{ github.workspace }}/sample-
app/src/main/java/anthos/samples/bankofanthos/balancereader
```

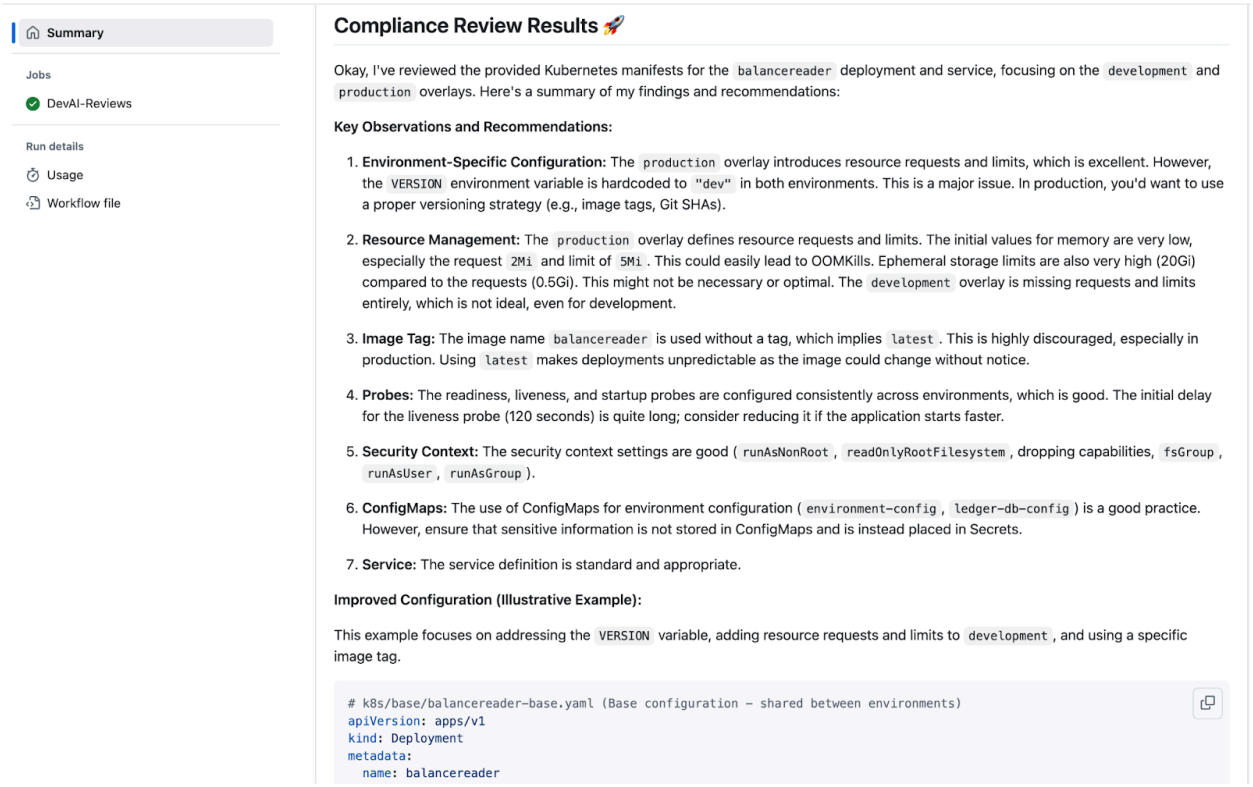
`devai review code` 指令會使用生成式 AI 模型，對提供的程式碼片段進行全面程式碼審查。這項工具會將待審查的程式碼做為輸入內容 (背景資訊)，並將輸出格式偏好設定做為輸出內容。這項工具會運用大型語言模型分析程式碼，判斷是否正確、有效率、容易維護、安全無虞，以及是否符合最佳做法。這項指令會建構詳細的提示詞，指示 Gemini 如何進行審查，然後傳送至模型來評估提供的程式碼。最後，它會處理 Gemini 的回覆，並根據使用者的偏好設定，將回覆格式化為 Markdown、JSON 或表格，然後輸出評論結果。



法規遵循審查指令的結果：

```
devai review compliance -c ${{ github.workspace }}/sample-app/k8s --config ${{ github.workspace }}/devai-cli/gemini/styleguide.md
```

`devai review compliance` 指令會根據一組最佳做法 (通常是 Kubernetes 設定) 分析程式碼。這項指令會使用 Gemini 模型檢查提供的程式碼 (`context`)，並與另一個設定檔 (`config`) 中定義的標準進行比較。這項指令會利用提示，指示 Gemini 模型扮演 Kubernetes 專家工程師，並提供合規報告。然後將發現項目格式化為簡要說明，重點在於程式碼範例，示範如何解決任何已識別的問題。最後，這項指令會將 Gemini 的法規遵循審查輸出內容列印到控制台。方便使用者輕鬆稽核程式碼是否符合規定。


Summary
Jobs
DevAI-Reviews
Run details
Usage
Workflow file
Compliance Review Results 🚀
Okay, I've reviewed the provided Kubernetes manifests for the balancereader deployment and service, focusing on the development and production overlays. Here's a summary of my findings and recommendations:
Key Observations and Recommendations:
1. Environment-Specific Configuration: The production overlay introduces resource requests and limits, which is excellent. However, the VERSION environment variable is hardcoded to "dev" in both environments. This is a major issue. In production, you'd want to use a proper versioning strategy (e.g., image tags, Git SHAs).
2. Resource Management: The production overlay defines resource requests and limits. The initial values for memory are very low, especially the request 2Mi and limit of 5Mi. This could easily lead to OOMKills. Ephemeral storage limits are also very high (20Gi) compared to the requests (0.5Gi). This might not be necessary or optimal. The development overlay is missing requests and limits entirely, which is not ideal, even for development.
3. Image Tag: The image name balancereader is used without a tag, which implies latest. This is highly discouraged, especially in production. Using latest makes deployments unpredictable as the image could change without notice.
4. Probes: The readiness, liveness, and startup probes are configured consistently across environments, which is good. The initial delay for the liveness probe (120 seconds) is quite long; consider reducing it if the application starts faster.
5. Security Context: The security context settings are good (runAsNonRoot , readOnlyRootFilesystem , dropping capabilities, fsGroup , runAsUser , runAsGroup).
6. ConfigMaps: The use of ConfigMaps for environment configuration (environment-config , ledger-db-config) is a good practice. However, ensure that sensitive information is not stored in ConfigMaps and is instead placed in Secrets.
7. Service: The service definition is standard and appropriate.
Improved Configuration (Illustrative Example):
This example focuses on addressing the VERSION variable, adding resource requests and limits to development , and using a specific image tag.
k8s/base/balancereader-base.yaml (Base configuration - shared between environments)
apiVersion: apps/v1
kind: Deployment
metadata:
name: balancereader

步驟 10 · 約 0 分鐘

10. 複製存放區

返回 Cloud Shell 終端機，然後複製存放區。

執行指令前，請將 `YOUR-GITHUB-USERID` 變更為 GitHub 使用者 ID。

```
git clone https://github.com/YOUR-GITHUB-USERID/genai-for-developers.git
```

變更資料夾，並在 Cloud Shell 編輯器中開啟工作流程檔案。

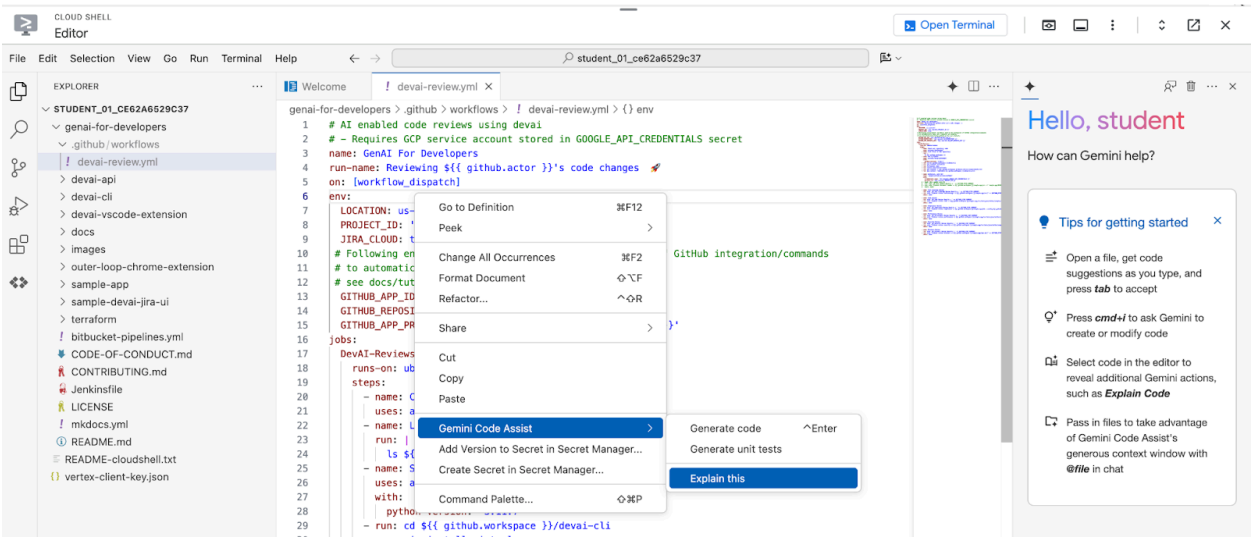
```
cd genai-for-developers

cloudshell edit .github/workflows/devai-review.yml
```

等待 IDE 顯示設定檔。

步驟 11 · 約 0 分鐘

11. 使用 Gemini Code Assist 解釋程式碼

在 `devai-review.yml` 檔案的任意處按一下滑鼠右鍵，然後選取 `Gemini Code Assist > Explain`。



審查說明：

Gemini

Okay, let's break down the devai-review.yml file, which seems to be a GitHub Actions workflow for performing AI-powered code reviews using a tool called "devai."

Overall Purpose

This workflow is designed to automatically review code changes within a repository using the "devai" command-line interface (CLI) tool. It performs several types of reviews, including:

- **Test Coverage:** Checks if there are sufficient tests.
- **Code:** Analyzes code quality, style, and potential issues.
- **Compliance:** Checks if the code adheres to certain style guidelines or security standards.
- **Performance:** Assesses the potential performance of the code.
- **Security:** Looks for security vulnerabilities.
- **Blockers:** Identifies critical issues that might prevent the code from working correctly or being merged.
- **Docs:** Checks and update the project documentation

Workflow Trigger

- on: [workflow_dispatch] : This means the workflow is triggered manually from the GitHub interface.

Environment Variables

- LOCATION: us-central1 : Specifies the GCP region.
- PROJECT_ID: '${{ secrets.PROJECT_ID }}' : The GCP project ID, retrieved from GitHub secrets.
- JIRA_CLOUD: true : Indicates the usage of Jira Cloud, potentially for integration.
- GITHUB_APP_ID , GITHUB_REPOSITORY , GITHUB_APP_PRIVATE_KEY : These are configured to open pull request automatically.

Jobs

- DevAI-Reviews : This is the main job, containing all the steps for the code review.
  - runs-on: ubuntu-latest : The job will run on the latest version of Ubuntu.

Steps

Let's break down the steps within the DevAI-Reviews job:

1. **Checkout Code:**

步驟 12 · 約 0 分鐘

12. DevAI CLI 開發

在本節中，您將變更 `devai cli`。

首先，請設定 Python 虛擬環境、安裝必要條件，然後執行範例指令。

```
cd ~/genai-for-developers/devai-cli
python3 -m venv venv
. venv/bin/activate
pip3 install -r src/requirements.txt
pip3 install --editable ./src
devai echo
```

設定必要環境變數。

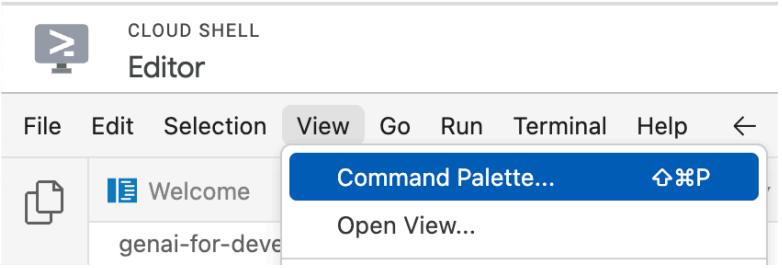
```
export PROJECT_ID=$(gcloud config get-value project)
export LOCATION=us-central1
```

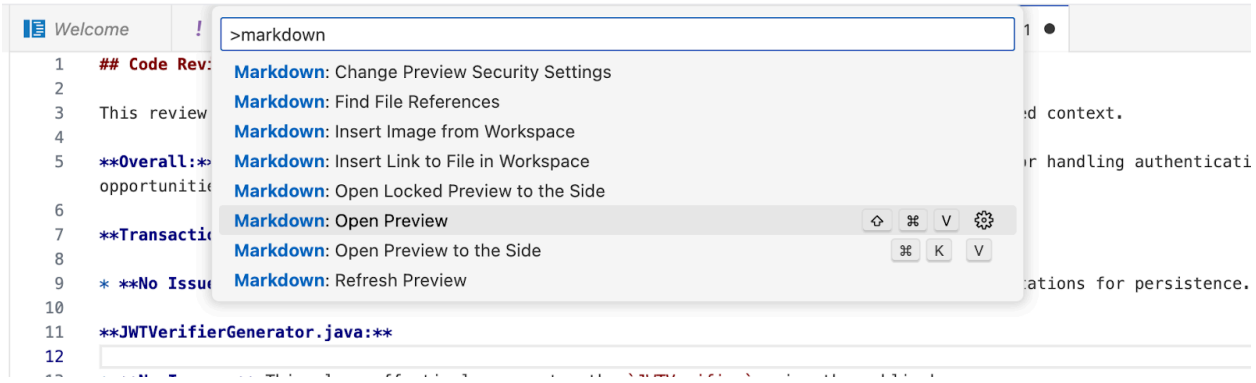
執行程式碼審查指令，確認一切運作正常：

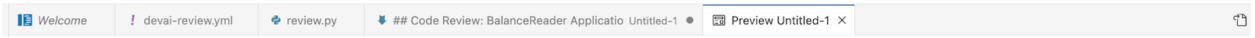
```
devai review code -c ~/genai-for-developers/sample-
app/src/main/java/anthos/samples/bankofanthos/balancereader > code-review.md

cloudshell edit code-review.md
```

在 Cloud Shell 編輯器中使用 Markdown 預覽功能查看結果。

使用指令區塊面板，然後選取「`Markdown: Open Preview`」。







Code Review: BalanceReader Application

This review focuses on the Java code for the BalanceReader application, considering the provided context.

Overall: The code generally appears well-structured and implements common best practices for handling authentication, background tasks, and caching. However, there are opportunities for improvement in error handling, potential race conditions, and code clarity.

Transaction.java:

- **No Issues:** The `Transaction` class is a straightforward data model with appropriate annotations for persistence.

JWTVerifierGenerator.java:

- **No Issues:** This class effectively generates the `JWTVerifier` using the public key.

BalanceReaderController.java:

- **Potential Race Condition (Medium):** The `startWithCallback` method initializes the `latestTransactionId` and starts the background thread. There's a potential race condition where the thread might start processing transactions before the initial `latestTransactionId` is correctly set, potentially leading to missed transactions. Consider using a synchronization mechanism (e.g., a `CountDownLatch`) to ensure the thread starts only after the initial ID is set.
- **Error Handling (Medium):** In the `liveness` and `getBalance` methods, the error messages logged are generic (e.g., "Cache error"). Providing more specific error messages, potentially including the underlying exception message, would greatly aid in debugging and monitoring.
- **Cache Key (Low):** The cache key is solely the `accountId`. While this might be sufficient in the current context, it could lead to collisions if accounts from different banks (using the same account numbering scheme) are ever used. Consider incorporating the `routingNum` into the cache key for robustness.

TransactionRepository.java:

- **No Issues:** The repository interface defines clear queries.

LedgerReader.java:

- **Error Handling (Medium):** Similar to `BalanceReaderController`, improve error messages in `startWithCallback` and `getLatestTransactionId` methods for better debugging.
- **Poll Frequency (Low):** The `pollMs` value is hardcoded to 100ms. Consider making this configurable via environment variables or configuration files to allow for tuning based on load and desired latency.
- **Synchronization (Low):** While not explicitly mentioned, if multiple instances of `LedgerReader` are possible, ensure appropriate synchronization mechanisms (e.g., database-level locking) are in place when reading and updating the `latestTransactionId` to avoid race conditions.

BalanceReaderApplication.java:

- **Logging (Low):** The use of `Level.forName("STARTUP", Level.FATAL.intLevel())` is unconventional. Consider using standard log levels (e.g., `INFO` or a custom level) for clarity.
- **Error Handling (Low):** The application exits immediately if an expected environment variable is not found. While this is acceptable, consider logging the missing variable and gracefully shutting down to allow for monitoring and easier debugging in containerized environments.

BalanceCache.java:

步驟 13 · 約 0 分鐘

13. 探索 devai CLI 指令

測試涵蓋範圍檢查指令

```
devai review testcoverage -c ~/genai-for-developers/sample-app/src > testcoverage.md

cloudshell edit testcoverage.md
```

法規遵循審查指令

```
devai review compliance --context ~/genai-for-developers/sample-app/k8s --config ~/genai-for-developers/devai-cli/.gemini/styleguide.md > k8s-review.md

cloudshell edit k8s-review.md
```

成效回顧指令

```
devai review performance -c ~/genai-for-developers/sample-app/src/main/java > performance-review.md

cloudshell edit performance-review.md
```

安全審查指令

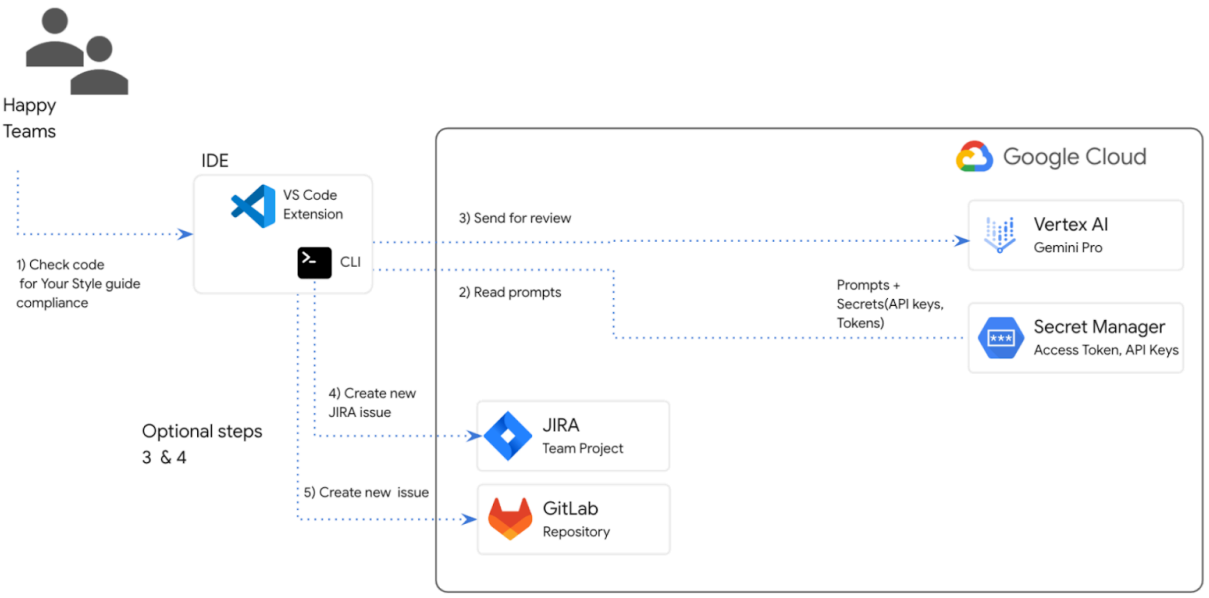
```
devai review security -c ~/genai-for-developers/sample-app/src/main/java > security-review.md

cloudshell edit security-review.md
```

封鎖者審查指令

```
devai review blockers -c ~/genai-for-developers/sample-app/pom.xml
devai review blockers -c ~/genai-for-developers/sample-app/setup.md
```

圖片/圖表審查和摘要

輸入圖表 [`~/genai-for-developers/images/extension-diagram.png`]：



查看指令：

```
devai review image \
  -f ~/genai-for-developers/images/extension-diagram.png \
  -p "Review and summarize this diagram" > image-review.md

cloudshell edit image-review.md
```

輸出內容：

```
The diagram outlines a process for conducting local code reviews using a VS Code extension or CLI, leveraging Google Cloud's Vertex AI (Gemini Pro) for generating review prompts.

**Process Flow:**

1. **Code Style Check:** Developers initiate the process by checking their code for adherence to pre-defined style guidelines.
2. **Prompt Generation:** The VS Code extension/CLI sends the code to Vertex AI (Gemini Pro) on Google Cloud.
3. **Vertex AI Review:**  Vertex AI analyzes the code and generates relevant review prompts.
4. **Local Review:** The prompts are sent back to the developer's IDE for their consideration.
5. **Optional Actions:** Developers can optionally:
    - Create new JIRA issues directly from the IDE based on the review prompts.
    - Generate new issues in a GitLab repository.

**Key Components:**

* **VS Code Extension/CLI:** Tools facilitating the interaction with Vertex AI and potential integrations with JIRA and GitLab.
* **Vertex AI (Gemini Pro):**  Google Cloud's generative AI service responsible for understanding the code and generating meaningful review prompts.
* **Google Cloud Secret Manager:** Securely stores API keys and access tokens required to authenticate and interact with Google Cloud services.
* **JIRA/GitLab (Optional):** Issue tracking and project management tools that can be integrated for a streamlined workflow.

**Benefits:**

* **Automated Review Assistance:**  Leveraging AI to generate review prompts saves time and improves the consistency and quality of code reviews.
* **Local Development:** The process empowers developers to conduct reviews locally within their familiar IDE.
* **Integration Options:** The flexibility to integrate with project management tools like JIRA and GitLab streamlines workflow and issue tracking.
```

圖片差異分析

```
devai review imgdiff \
  -c ~/genai-for-developers/images/devai-api.png \
  -t ~/genai-for-developers/images/devai-api-slack.png > image-diff-review.md

cloudshell edit image-diff-review.md
```

輸出內容：

```
The following UI elements are missing in the "AFTER UPGRADE STATE" image compared to
the "BEFORE UPGRADE STATE" image:

1. **Slack:** The entire Slack element, including the icon, "Team channel" label, and
the arrow indicating interaction, is absent in the AFTER UPGRADE image.
2. **Storage Bucket:**  The "Storage Bucket" element with its icon and "PDFs" label
is missing in the AFTER UPGRADE image.
3. **"GenAI Agents" label in Vertex AI block:** The BEFORE UPGRADE image has "Vertex
AI Agents" and "GenAI Agent" labels within the Vertex AI block, while the AFTER
UPGRADE image only has "Vertex AI."
4. **"Open JIRA Issue" and "Team Project" labels:** In the BEFORE UPGRADE image,
these labels are connected to the JIRA block with an arrow. These are missing in the
AFTER UPGRADE image.

**Decision Explanation:**

The analysis is based on a direct visual comparison of the two provided images,
noting the presence and absence of specific UI elements and their associated labels.
The elements listed above are present in the BEFORE UPGRADE image but absent in the
AFTER UPGRADE image.
```

影片檔案分析：

```
devai review video \
  -f "/tmp/video.mp4" \
  -p "Review user journey video and create unit tests using jest framework"
```

文件生成指令

```
devai document readme -c ~/genai-for-developers/sample-app/src/main/
```

輸出內容：

```
# Bank of Anthos - Balance Reader Service

## Description

The Balance Reader service is a component of the Bank of Anthos sample application.
It provides a REST endpoint for retrieving the current balance of a user account.
This service demonstrates key concepts for building microservices with Spring Boot
and deploying them to a Kubernetes cluster.

## Features
...
```

在 Cloud Shell 編輯器中查看可用的 devai 指令列指令：

```
cloudshell edit ~/genai-for-developers/devai-cli/README.md
```

或查看 GitHub 存放區中的 README.md。

步驟 14 · 約 0 分鐘

14. 恭喜！

恭喜，您已完成本程式碼研究室！

涵蓋內容：

- 在 GitHub 中新增生成式 AI 程式碼審查自動化步驟
- 在本機執行 devai CLI

後續步驟：

- 更多實務課程即將推出！

清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本教學課程所用資源的費用，請刪除含有相關資源的專案，或者保留專案但刪除個別資源。

刪除專案

如要避免付費，最簡單的方法就是刪除您為了本教學課程所建立的專案。